

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC2233 — Programación Avanzada 2026-2

Publicación: 12 de noviembre de 2026
Actividad evaluada

Actividad 9
Tópicos Avanzados

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: `Actividades/AC9`
- **Fecha máxima de entrega:** 12 de noviembre a las 17:10 hrs
- **Ejecución de actividad**: La Actividad será ejecutada **únicamente** desde la terminal del computador. **Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.**

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (***push***), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Introducción

En esta actividad estudiaremos el comportamiento de dos algoritmos de búsqueda en grafos: **Breadth-First Search (BFS)** y **Depth-First Search (DFS)**.

Para ello, utilizaremos un generador de laberintos que permite crear grafos de distintos tamaños y densidades. Nuestro objetivo será comparar el **tiempo de ejecución** y el **largo de las soluciones** encontradas por ambos algoritmos, utilizando **Pandas** para procesar los resultados y **Matplotlib** para visualizarlos.

Se entrega código base con la implementación de los grafos, el generador de laberintos y los algoritmos BFS y DFS. No es necesario modificar el código base, pero siéntete libre de hacerlo si prefieres o si quieres crear nuevos archivos. Esta actividad debe poder ejecutarse con el comando `python3 main.py`.

Antes de empezar, confirma que tengas instalado pandas y matplotlib. Sigue las siguientes instrucciones basado en el sistema que estés usando para trabajar:

- **ISO lubuntu**: Desde la carpeta `Actividades/AC9` ejecuta el comando `source install.sh`
- **SEB (MAC)**: Ejecuta el comando `python3 -m pip install pandas matplotlib`

Parte 1. Experimentación y recolección de datos

En esta parte deberás ejecutar BFS y DFS sobre distintos laberintos, medir sus resultados y almacenarlos utilizando Pandas.

Para realizar los experimentos, iremos variando el tamaño del laberinto y la densidad del laberinto (número de muros). Deberás usar las siguientes configuraciones:

- **Tamaños**: 10, 20, 30, 40, 50, 60, 70, 80, 90 y 100.
- **Densidades**: 0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9 y 1.0.
- **Repeticiones**: 5 por cada combinación de tamaño y densidad.
- **Algoritmos**: BFS y DFS.

Para cada combinación de tamaño y densidad, deberás generar 5 laberintos distintos y ejecutar ambos algoritmos sobre cada uno de ellos.

En cada ejecución deberás:

1. Generar un laberinto utilizando `generar_laberinto` con la combinación de densidad y tamaño correspondiente. Esto retornará un `namedTuple Laberinto` que contiene los valores `grafo`, `inicio` y `meta`.
2. Ejecutar BFS y DFS para encontrar un camino desde el inicio hasta la meta dados por el paso anterior.
3. Medir el tiempo en milisegundos que demora cada algoritmo utilizando la librería `time`. La medición debe considerar únicamente la ejecución del algoritmo de búsqueda, sin incluir la generación del laberinto. Ejemplo:

```
tiempo_inicio = time.perf_counter()
camino = bfs(grafo, inicio, meta)
tiempo_fin = time.perf_counter()

tiempo_segundos = tiempo_fin - tiempo_inicio
tiempo_milisegundos = tiempo_segundos * 1000
```

4. Obtener el largo de la solución encontrada, lo que corresponde a la cantidad de movimientos necesarios para llegar a la meta, o `len(resultado) - 1`.
5. Almacenar los resultados en un `DataFrame` de `pandas` con las siguientes columnas:

Columna	Descripción
tamano	Tamaño del laberinto
densidad	Densidad del laberinto
algoritmo	Algoritmo utilizado (`BFS` o `DFS`)
tiempo	Tiempo de ejecución en milisegundos
largo	Cantidad de movimientos de la solución encontrada

Importante: BFS y DFS deben ejecutarse sobre el mismo laberinto en cada repetición, para que sus resultados sean comparables.

Al finalizar esta parte, el DataFrame deberá contener **1100 filas**, correspondientes a todas las ejecuciones realizadas.

Parte 2. Procesamiento de datos con Pandas

Una vez obtenidos los resultados, deberán utilizar `pandas` para calcular estadísticas que permitan comparar el comportamiento de los algoritmos.

Para esto debes:

1. Agrupar los datos según las columnas `tamano`, `densidad` y `algoritmo`.
2. Calcular el **tiempo promedio de ejecución** para cada grupo.
3. Calcular el **largo promedio de las soluciones** para cada grupo.
4. Almacenar los resultados en un nuevo `DataFrame` con las siguientes columnas:

Columna	Descripción
tamano	Tamaño de laberintos agrupados
densidad	Densidad de laberintos agrupados
algoritmo	Algoritmo utilizado por el grupo (`BFS` o `DFS`)
tiempo_promedio	Promedio de los tiempos de ejecución en milisegundos
largo_promedio	Promedio de los largos de las soluciones

Importante: El nuevo `DataFrame` deberá contener **220 filas**, correspondientes a las distintas combinaciones de tamaño, densidad y algoritmo.

Tip: Revisa la documentación de Pandas que se agregó a `Syllabus/Archivos importantes`

Parte 3. Visualización de resultados con `matplotlib`

Utilizando el `DataFrame` procesado en la sección anterior, tu programa deberá generar **8 gráficos independientes** con `matplotlib` para comparar el comportamiento de BFS y DFS. Estos deberán ser guardados como archivos en la carpeta graficos. Para esto en vez de usar el método `plt.show()`, debes usar `plt.savefig(path.pdf)`.

Cada gráfico deberá mostrar los resultados de ambos algoritmos, utilizando colores diferentes para distinguirlos.

Parte 3.1. Tiempo promedio de ejecución

Selecciona **4 densidades distintas** de entre los valores utilizados en los experimentos.

Para cada densidad seleccionada, genera un gráfico de dispersión utilizando `plt.scatter()` que muestre cómo varía el tiempo promedio de ejecución según el tamaño del laberinto.

Cada gráfico deberá contener:

- **Eje X**: Todos los tamaños utilizados en los experimentos, además de un `xlabel`.
- **Eje Y**: Tiempo promedio de ejecución en milisegundos además de un `ylabel`.
- **Dos conjuntos de puntos**: Uno para BFS y otro para DFS, utilizando colores diferentes. Debe existir además una leyenda que indique qué color corresponde a cada algoritmo.
- **Título**: La densidad correspondiente al gráfico.

Parte 3.2. Largo promedio de las soluciones

Selecciona **4 tamaños distintos** de entre los valores utilizados en los experimentos.

Para cada tamaño seleccionado, genera un gráfico de dispersión utilizando `plt.scatter()` que muestre cómo varía el largo promedio de las soluciones según la densidad del laberinto.

Cada gráfico deberá contener:

- **Eje X**: Todas las densidades utilizadas en los experimentos, además de un `xlabel`.
- **Eje Y**: Largo promedio de las soluciones, medido en movimientos, además de un `ylabel`.
- **Dos conjuntos de puntos**: Uno para BFS y otro para DFS, utilizando colores diferentes. Debe existir además una leyenda que indique qué color corresponde a cada algoritmo.
- **Título**: El tamaño correspondiente al gráfico.

Bonus

Puedes obtener puntos adicionales en la actividad con los siguientes bonus:

Bonus 1: Visualización en tres dimensiones (1 punto)

Genera un gráfico tridimensional utilizando `matplotlib` para visualizar cómo varía el tiempo promedio de ejecución según el tamaño y la densidad del laberinto.

Para esto, selecciona uno de los dos algoritmos (BFS o DFS) y construyan un gráfico de dispersión en tres dimensiones que contenga:

- **Eje X**: Tamaño del laberinto. Debe incluir un `xlabel`
- **Eje Y**: Densidad del laberinto. Debe incluir un `ylabel`
- **Eje Z**: Tiempo promedio de ejecución en milisegundos. Debe incluir un `zlabel`
- **Conjunto de puntos**: Debe existir una leyenda que indica a qué corresponde el conjunto de puntos.

El gráfico deberá utilizar todas las combinaciones de tamaños y densidades experimentadas para el algoritmo seleccionado.

Bonus 2: Conclusiones (1 punto)

Analizando los datos a través de los gráficos obtenidos, contesta las siguientes preguntas sobre los algoritmos en un archivo `README.md`:

1. ¿Cómo afecta la densidad de un laberinto a DFS y BFS?
2. ¿Cómo afecta el tamaño de un laberinto a DFS y BFS?

Considere tanto el tiempo de ejecución como el tamaño de la solución.

Notas

- Sube todos los archivos del programa al repositorio. Puedes modificar todos los archivos del programa.
- Tu programa deberá ejecutarse por consola con el comando `python3 main.py`
- No subas archivos de gráficos al repositorio.

Puntaje por sección

Los criterios indican los aspectos principales que serán evaluados y pueden subdividirse durante la corrección.

Criterio	Puntaje
Parte 1: Experimentación y recolección de datos.	
Experimentación. Se recorren todas las combinaciones de tamaños, densidades y repeticiones solicitadas. Se ejecutan BFS y DFS sobre los mismos laberintos, obteniendo correctamente el tiempo de ejecución y el largo de las soluciones.	1,0 pts
Almacenamiento de datos. Los resultados de todas las ejecuciones se almacenan correctamente en un DataFrame con las columnas solicitadas.	1,0 pts
Parte 2: Procesamiento de datos con Pandas.	
Se agrupan correctamente los datos según tamaño, densidad y algoritmo. Se calculan los promedios de tiempo y largo de las soluciones, almacenándolos en un nuevo DataFrame con las columnas solicitadas.	1,0 pts
Parte 3: Visualización de resultados con Matplotlib.	
Tiempo promedio de ejecución. Se generan correctamente los 4 gráficos solicitados para comparar el tiempo promedio de BFS y DFS según el tamaño del laberinto.	1,0 pts
Largo promedio de las soluciones. Se generan correctamente los 4 gráficos solicitados para comparar el largo promedio de las soluciones de BFS y DFS según la densidad del laberinto.	1,0 pts
Bonus 1: Visualización en tres dimensiones.	
Se genera correctamente un gráfico tridimensional que muestra el tiempo promedio de ejecución en función del tamaño y la densidad del laberinto para uno de los algoritmos, utilizando todas las combinaciones experimentadas.	+1,0 pts
Bonus 2: Conclusiones.	
El alumno llega a una conclusión correcta basado en los gráficos generados .	+1,0 pts
Descuentos.	
Se podrá aplicar un máximo de **1 punto de descuento** por código con errores que impidan su ejecución, problemas de legibilidad, incumplimiento de PEP 8 o de las restricciones indicadas.	
Total	**5,0 pts**
Total con bonus	**7,0 pts**